Generative AI and Developer Workflows: How GitHub Copilot and ChatGPT Influence Solo and Pair Programming

Viktoria Stray
University of Oslo, SINTEF
stray@uio.no

Nils Brede Moe
SINTEF
nilsm@sintef.no

Nivethika Ganeshan
University of Oslo
nivethig@uio.no

Simon Kobbenes
University of Oslo
simontko@uio.no

Abstract

This study investigates the impact of generative AI (GAI) tools, like ChatGPT and GitHub Copilot, on software development. GAI tools can automate repetitive tasks, provide code suggestions, and enhance problem-solving. We conducted a case study in a large-scale agile organization, where we examined how GAI tools influence development efficiency and developers' workflow. During 25 days of observations, we observed 49 programming sessions. GAI was used in all 16 solo programming sessions and 30 of 33 pair programming sessions. Additionally, we conducted 14 interviews with developers and team leaders. In solo programming, GAI tools were said to improve efficiency and reduce stress, allowing more time for solving complex tasks. In contrast, the impact of GAI in pair programming was mixed. GAI tools were less frequently used in pair programming compared to solo programming. Our findings also highlight changes in software development, with developers leveraging GAI for information retrieval and coding assistance. However, challenges such as dependency on AI tools and the accuracy of AI-generated suggestions persist.

Keywords: Agile software development, Developer productivity, Team collaboration, Case study

1. Introduction

Software engineering has evolved significantly to incorporate new technological advancements and meet modern business requirements. A recent advancement is the adoption of generative AI (GAI), which has transformed how developers gather information, solve tasks, learn, and collaborate (Ulfsnes et al., 2024). GAI, particularly tools like ChatGPT and GitHub Copilot, have been rapidly adopted due to their ability to improve productivity and streamline workflows (Brynjolfsson et al., 2023; Nguyen-Duc et al., 2023). GAI tools assist developers by providing intelligent code suggestions, automating repetitive tasks, and enhancing problem-solving capabilities (Dohmke, 2023; Imai, 2022). Further, ChatGPT, developed by OpenAI, engages users in natural language conversations, offering solutions and suggestions for various coding tasks (Surameery & Shakor, 2023).

Several studies have explored the effects of GAI on development practices. For example, Cabrero-Daniel et al. (2024) recently examined the integration of AI assistants in agile software development and identified critical drivers of success and the link between AI use and team collaboration dynamics. Similarly, Zhang et al. (2023) reported that developers experienced improvements in their coding practices. Furthermore, a recent survey found that GitHub Copilot users reported a significantly enhanced ability to focus on engaging tasks compared to non-users (Wivestad et al., 2024).

Understanding how GAI influences collaboration dynamics is crucial for optimizing coding practices in modern software development environments. Further, there is a call for more research on the human-centered aspects of GAI (Russo et al., 2024). Therefore, we ask the following question:

RQ: *How has the integration of generative AI tools influenced a developer's workday?*

To address this question, we collected and analyzed qualitative data from observations and interviews with developers and team leaders in a large Fintech company. Our study aims to understand developers' experiences with programming in the context of GAI, highlighting the challenges, benefits, and potential risks associated with using GitHub Copilot and ChatGPT.

2. Background

In the dynamic field of software development, the integration of generative AI (GAI) tools represents a significant advancement. These tools have revolutionized how developers perform tasks, solve

Accepted paper (In Press). Cite as: Stray, V., Moe, N.B., Ganeshan, N., Kobbenes, S. "Generative AI and Developer Workflows: How GitHub Copilot and ChatGPT Influence Solo and Pair Programming", In Proceedings of the 58th Hawaii International Conference on System Sciences, 2025

problems, and collaborate. This section provides an overview of the background relevant to understanding the integration and impact of GAI tools within the software development workflow.

2.1. Developer Workflows and Activities

Understanding developers' workflows is critical to improving their efficiency and overall productivity. Previous studies have focused on how developers allocate their time across different tasks. Meyer et al. (2019) classified the tasks into three groups: *development-heavy*, *collaboration-heavy*, and *other activities*. Development-heavy tasks include coding, bug fixing, testing, and documentation, while collaboration-heavy tasks involve meetings, emails, and supporting colleagues. Other activities encompass learning, administrative tasks, and breaks. While developers dedicate much of their time to development-heavy activities, the reported time spent on these tasks varies significantly (Astromskis et al., 2017; Meyer et al., 2019).

Communication, coordination, and social connections are essential components of team culture in a development context. These elements play a crucial role in shaping team dynamics and enhancing individual developers' experience, especially in large-scale agile (Berntzen et al., 2023). Research has also shown a strong connection between well-being, job satisfaction, and productivity (Miller et al., 2021). Therefore, it is significant to know how developers socialize, communicate, and support each other. Teams in large-scale agile environments often rely on structured coordination practices, which facilitate giving and receiving help across team boundaries (Gustavsson et al., 2022). This collaborative aspect of team culture can have an important impact on the work environment and individual performance. For instance, Gonçalves et al. (2011) found that developers spend 45% of their time engaged in collaborative activities, outweighing development-heavy tasks. Further research suggests that this percentage may be even higher in some cases (Chattopadhyay et al., 2021). Additionally, a recent study found that software engineering practitioners spend half their workday in meetings (Stray et al., 2024), emphasizing the importance of collaboration and coordination in modern development workflows.

2.2. Generative AI in Software Development

The emergence of GAI tools in software development has been driven by media coverage and the increasing capabilities of these tools. Developers initially adopted tools like ChatGPT in private settings, often influenced by its popularity and perceived potential (Ulfsnes et al., 2024). As developers became more familiar with these tools, they integrated them into their professional workflows. These tools have been rapidly adopted due to their ability to improve productivity and streamline workflows. However, the adoption process has not been without challenges. Developers have expressed concerns about the accuracy and reliability of GAI-generated suggestions, data security, and the potential for dependency on these tools.

Further examination of Copilot's capabilities indicates a significant proficiency in generating valid code, with a success rate of 91.5% (Yetistiren et al., 2022). However, the same study found that there was a noteworthy variation in code correctness, with 28.7% of problems solved correctly, 51.7% solved partially correctly, and 20.1% solved incorrectly. These results varied depending on the precision of the developer's prompts and methods used. By not using prompts and only using function names or dummy function names, the success and validity rate decreased significantly. While using Copilot, using proper explanations of the given problem is very important for acquiring the correct and valid code.

Further, developers now rely on AI-driven solutions for tasks previously handled through peer collaboration or extensive online searches (Ulfsnes et al., 2024). This shift has implications for how coding practices are performed and how developers interact with their tools and peers. ChatGPT has been found to often surpasses human experts and state-of-the-art methods with its detailed and credible responses for software engineering tasks but occasionally also produces incorrect answers, rendering it unsuitable for many such tasks (Sridhara et al., 2023).

3. Method

We decided to conduct a case study since *"A case study is an empirical inquiry that investigates a contemporary phenomenon within its real-life context, especially when the boundaries between phenomenon and context are not clearly evident."* (Yin, 2003, p. 13). We chose a *qualitative approach* because of its capability to investigate a phenomenon in-depth through words and actions. They are applicable for gaining the developers' understanding and perception regarding the phenomena, as well as their reflection on the research question. Therefore, a qualitative approach is well-suited for generating rich insights (Sofaer, 1999). Given the exploratory nature of case studies (Yin, 2003), they are particularly appropriate for investigating

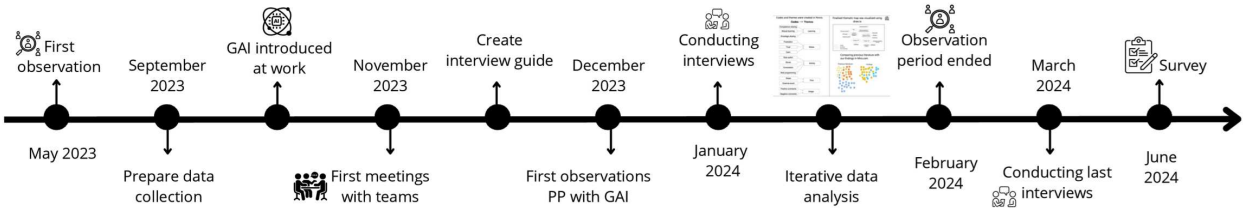

Figure 1. Timeline of data collection and analysis

phenomena that are underexplored in the literature.

We observed and interviewed developers in the large-scale software development organization Sparebank1 Utvikling. It comprises approximately 600 employees, including full-time employees and external consultants. The organization is dedicated to serving individual clients and businesses. The implementation of GAI was relatively new and was introduced during our observation period. Pair programming was a well-established practice used by most teams. We also had access to a company-wide Slack channel.

The observations and semi-structured interviews were conducted in a period spanning from May 2023 to March 2024. In June 2024, we conducted a small survey among the studied teams on the time saved by using GAI. The timeline of our data collection and analysis is shown in Figure 1.

Our observations involved two teams. Team 1 had 17 members in three subteams. In total, they were 10 developers, 3 designers, 2 testers, 1 product owner, and 1 team leader. Team 2 had 11 members, which included 5 developers, 1 designer, 1 tester, 1 product owner, 1 team leader, 1 architect, and 1 system administrator

We were able to closely observe the teams in the office area where the team worked, although desk assignments occasionally varied based on availability. On most observation days, we had two researchers in the company, which was beneficial. As emphasized by Sharp et al. (2019, p. 291), collaborative observation allows observers to focus on different individuals or parts of the context, thereby covering more ground. The presence of more than one observer can facilitate a more seamless integration of observation and reflection. As a result, we decided to watch various people and elements within the teams. Despite the team member's hectic workdays, we managed to collect a substantial amount of empirical data through observations.

We collected observational data over a period of seven weeks: 2 working days in May 2023, 8 in December 2023, 9 in January 2024, and 6 in February 2024, a total of 25 days. Conducting observations before interviewing was valuable to get a deeper understanding and evaluation of the interviewee's response. It allowed the interviewees to clarify and discuss on specific events or statements from the observations. One additional benefit of having the interviews after the observation was that it facilitated a relaxed conversation since they had already met us during the observation period. Sharp et al. (2019, p.268) have described interviews as "conversation with a purpose".

Before conducting the interviews, we developed an interview guide based on the research question and data gathered from observations. A well-structured interview guide is a valuable tool for ensuring that all relevant steps are followed and that no important topics are overlooked (Lazar Jonathan, 2017, p.210). The guide included questions such as, "How have AI tools impacted your workflow and your role as a developer?", "How do you evaluate the solutions or suggestions provided by the AI tool?", and "What challenges have you encountered when using AI in development tasks?". In total, we conducted 14 interviews between January and March 2024. The first five interviews (I1, I2, I3, I4, and I5) took place in the winter, while the remaining interviews (I6–I14) were held in the spring. Participants included members from both teams.

We used *open coding* (Lazar Jonathan, 2017, p. 306) as the initial technique in our data analysis. We let the themes emerge naturally from the data itself (Sharp et al., 2019, p. 322). The data analysis was an iterative process, and later we adopted a more systematic approach in order to identify themes and group the themes into factors shaping a good workday for developers (Meyer et al., 2019).

4. Results

This section presents the results of our thematic analysis on integrating generative AI (GAI) in software development. We focus on the advantages, disadvantages, and implications of using GAI tools in daily developer work, their interaction with these tools, and their perceptions of their usefulness. The majority of interviewees, as presented in Table 1, highlighted the potential of GAI tools like Copilot to enhance efficiency in everyday tasks, particularly in

Table 1. Interviewees' opinions on the main advantages and disadvantages of using GAI.

Interviewee ID	Advantage	Disadvantage
I1	Enhances efficiency	Inaccurate answers
I2	Helpful when migrating code	Data security
I3	Better flow while developing	Inaccurate answers
I4	Enhances efficiency	Interrupts workflow
I5	Reduces work stress	Interrupts workflow
I6	Practical	Inaccurate answers
I7	Enhances efficiency	Inaccurate answers
I8	Supplementing knowledge	Inaccurate answers
I9	Enhances efficiency	Inaccurate answers
I10	Enhances efficiency	Inaccurate answers
I11	Enhancing efficiency	Interrupts workflow
I12	Helpful when migrating code	Interrupts workflow
I13	Enhance efficiency and better flow	Suboptimal code
I14	Access to information	Fear of automation impact/Job security

coding. The most mentioned disadvantage of GAI was inaccurate answers from the tools. The developers would frequently encounter inaccuracies and misleading information, which led to trust issues with the output generated by GAI. The developers spent much time verifying the AI-generated suggestions and considering privacy implications. When asked how much time they saved using the technology, the developers reported an average of 23 minutes per day. We observed 16 sessions where developers worked alone; GAI was used in all of these. We observed 31 physical pair programming sessions; GAI was used in 29 of these. We observed two remote pair programming sessions; GAI was used in one of these. However, although GAI was used in most programming sessions, it was used more frequently when programming alone.

4.1. The hype and adoption of generative AI

Following our observation in May 2023, we noticed that some developers were using ChatGPT to assist them when encountering problems. At that time, they appeared uncomfortable and embarrassed when asked about their usage. However, by January 2024, they were more open about how they used GAI individually and in collaborative processes. One developer mentioned, "*We have a license to use this sort of GAI.*"

The majority of the interviewees stated that they started using the GAI tools in private settings because of its media coverage and hype. I7 stated, *"it was all over the news[...] It was exciting to see its capabilities"*. Another developer noted, *"It started in my last year at university when ChatGPT gained popularity. I was initially skeptical, but I found it remarkably proficient, especially for coding."* I1 said, *"At the outset, with considerable discussion and perhaps even hype surrounding it, one starts to contemplate: Maybe I should catch this wave"*. Half of the interviewees mentioned that ChatGPT was the first GAI tool they tried. In addition, I1 and I7 mentioned that they had no formal training in how to use GAI in the organization, and I13 explained that their initial trial with GAI was a form of *"trial and error"*.

4.2. GAI-centric workflow

With the integration of GAI tools in the organization, the development workflow became AI-centric. The developers now used the tools in their workflow.

Many developers used the autocomplete function in Copilot. This meant that Copilot would provide a suggestion for the rest of a line of code as soon as a developer started writing code. In this way, Copilot completed unfinished lines of code and suggested context-specific snippets. This change in the workflow was seen as a benefit. One developer (I6) described, *"Smaller tasks are completed more efficiently, minimizing time spent on low-value activities."* In one observation, a developer with a high frequency of generative AI usage evaluated and integrated over 66 lines of code (LOC) within 15 minutes. Of these, 37 LOC were generated using *comments* within the IDE, five were completions of an *unfinished line of code*, and over 24 LOC were added based on context within the file, such as *function name*. These LOC were divided over 17 accepted suggestions, where the developer had to modify eight suggestions.

As a consequence of the new workflow, the developers wrote less code and spent more time reviewing suggestions. This worked fine as long as the suggestions were good. One day we observed the developers work, a developer was visibly and audibly frustrated, letting out a sigh when the AI failed to generate the correct suggestion. While autocomplete was sometimes frustrating, developers still appreciated this functionality. Developer I5 explained, *"Yes, it may sound like it is a little indifferent to having AI or not. But I know I can't work completely without AI. I like Copilot because of the easy and small autocompletes; I don't want it to suggest ready-made solutions, but just something little as helping with syntax."* Another developer, I7, stated, *"I could solve the task myself, but describing it to AI is quicker, yielding a solution faster."*

Searching for knowledge in an effective way is important for making progress in software development. The use of traditional search engines like Google and reading on forums like Stack Overflow declined as the developers preferred the convenience of AI tools for quick information retrieval. One developer (I4) mentioned, *"If you don't find it within the top three or four searches on Google, being more specific in further searches won't help. That's where ChatGPT*

shines." This shift to an AI-centric workflow changed their approach to problem-solving. During observations, there was minimal use of search engines, as highlighted by I5, who mentioned that *"It has more or less replaced Google searches, which I rarely use now"*. Within this context, I13 explained *"I believe it has had a positive impact on efficiency. It streamlines the workflow by eliminating the need to constantly switch back and forth to find answers or solutions"*. I3 further emphasized the personal gains from this change, *"It's a huge efficiency boost. Going back to Googling and browsing through docs would be a productivity setback for me"*.

While the developers found GAI to be helpful while developing code, we also examined the specific impact GAI had on pair programming (PP). During these observations, we found that there was a decrease in the interactions with GAI compared to when they were programming alone. We observed that the presence of a human partner resulted in the developer preferring human interactions and input over interacting with GAI. This observed trend is consistent with the experience shared by I4: *"My focus on Copilot diminishes when paired with a programming partner. Overall, I don't pay as much attention to Copilot's proposals"*. Adding to this, I11 mentioned that *"AI can be a bit of a disturbing element"* referring to the number of suggestions that appeared while developing.

Additionally, we observed that when they were pair programming, GAI was not favored as an information-gathering tool. When solving a problem, we observed that developers first asked their partner, then searched on internal Slack channels, then used Stack Overflow and Google, and finally used Chat GPT if they still could not solve the issue. If they came up with new ideas on how to solve the issue, they went back to searching externally.

4.3. Developer experience affecting GAI use

We observed that less experienced developers tended to rely more heavily on AI tools than the seniors. We also observed that less experienced developers often accepted functions and code snippets that could not be compiled, used incorrect variables, or lacked imports. For example, in one instance, a less experienced developer wasted time troubleshooting a function that could not be compiled. In contrast, experienced developers seemed to use these tools more selectively, leveraging their experience to verify and refine AI-generated code.

Moreover, this trend extended into PP sessions. One experienced developer (I2) said that *"It does not affect PP negatively. It makes me code faster, and I don't think it is disturbing at all"*. During PP sessions with less experienced developers, they took the time to consider and discuss the suggestions generated by Copilot. However, we observed that an experienced developer made a collective choice as a driver to use or not use the suggested code. This was also seen in other observations with an experienced and less experienced developer. The less experienced frequently accepted and discarded suggestions. In contrast, the more experienced appeared to utilize Copilot more strategically as an advanced autocomplete tool.

4.4. Cognitive load

When exploring how Copilot affects developers' workdays, the interviewees highlighted not just efficiency as a benefit but also Copilot's role in enhancing workplace well-being by reducing the stress associated with constant decision-making. I3 and I5 described that Copilot facilitated mental ease when faced with coding tasks where Copilot could suggest viable paths forward, which would reduce uncertainty, allowing developers to proceed with a greater sense of confidence and calm. I3 noted, *"If it can provide a good suggestion, I'll use it. It's a bit lazy of me"*. I5 affirmed this by stating:

> *"I'm not sure if I will complete tasks more quickly, but I definitely feel calmer about the work. It lightens the load significantly. Essentially, it's more about choosing a direction and sticking with it. So, while it certainly makes the process more comfortable, it doesn't necessarily mean I'm more efficient."*

4.5. A sparring partner

During our observations, developers frequently utilized ChatGPT and Copilot as sparring partners in their coding process. Developers often added comments to their code to guide Copilot in generating relevant suggestions, enhancing the accuracy and relevance of these suggestions before accepting them. Interviewee 14 described *"Especially if you get stuck, it often helps you move forward. I think it makes me learn much faster. It helps me get unstuck. All those simple things that developers often get stuck on, I don't get stuck on them anymore. So, I move on to the more difficult problems much faster. And you learn more from that."*

For example, we observed a developer added the comment *"check json response agreements length"* to assist Copilot in generating the correct code. In observation of a pair programming session, the

```go
client.go 1, U http/client.go/...
package http

// Create a http client function. Perform a get. Return a response and error.
func Get(url string, dto interface{}) (*http.Response, error) {
    resp, err := http.Get(url)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()

    return resp, nil
}
```

Figure 2. Copilot generating an incorrect suggestion based on user comment.

```go
client.go 1, U http/client.go/...
package http

// Create a http client function. Perform a get. Marshall json. Return a response and error.
func Get(url string, jsonDto *any) (*http.Response, error) {
    resp, err := http.Get(url)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    err = json.NewDecoder(resp.Body).Decode(jsonDto)
    if err != nil {
        return nil, err
    }
    return resp, nil
}
```

Figure 3. Copilot generating a refined suggestion based on improved user comment.

developer initially accepted a for loop suggested by Copilot, only to quickly remove it after realizing it was incorrect. They then added a guiding comment: *"check amount and product name for first agreement"*, which helped refine Copilot's output to produce 11 lines that fulfilled their specific needs. This process of guiding Copilot is illustrated in Figures 2 and 3. It shows that small changes in context can help Copilot generate new and improved suggestions. The developer explained that the code in Figure 2 was incorrect, necessitating a change in the comment.

Building on the issue of receiving incorrect or misleading information from GAI tools, we frequently observed that the developers rejected the suggestions provided by Copilot. However, not all incorrect suggestions were rejected. In some cases, developers accepted incorrect suggestions but spent time modifying these outputs to correct and adapt the code to their specific needs. This process was time-consuming and consisted of assessing, integrating, and revising AI-generated suggestions. For instance, a developer accepted an incorrect proposal, removed most of the lines, and used it as a template for further refinement rather than coding the function from scratch. This pattern was observed but not mentioned by any interviewees.

After accepting a suggestion from Copilot, developers often engaged in an iterative process to refine it. This interaction involved evaluating the suggestion by looking at code structure, length, complexity, and variable names to assess its usefulness. If it was considered valuable and provided a base, the developer would use the IDE to check syntax and verify if the code was executable. At this stage in the process, the developer would decide whether to keep the suggestion or discard it based on practicality. Sometimes, this process took several attempts. During an observation, a developer evaluated multiple LOCs suggested by Copilot, which were initially incorrect. The developer then created a class name, datatype, and a few variables before Copilot could generate the next LOC correctly. This operational dynamic was displayed in this insight from I11:

> *"Copilot is used for building on a function, when you write descriptive function names, you hope to see if it comes up with something that might be correct. And then you take it if it's on to something, and then*

either work on it further or discard it."

We observed that Copilot's suggestions did not need modification when its use was centered on completing an unfinished LOC and when assisting with syntactical constructions. I6 referred to Copilot as an *advanced autocomplete function*. However, we also observed that most suggestions by Copilot were skipped or rejected. Several interviewees mentioned that it often felt like it sometimes disrupted their coding workflow.

During our observations, we noticed that some developers preferred ChatGPT over Copilot. A developer who preferred ChatGPT provided context by copying code from the IDE into ChatGPT and modifying the code slightly. Then he described his expectations from ChatGPT before submitting the query. When receiving the suggestion, the developer pasted it back into the IDE. During the interview about this process, I14 highlighted the importance of providing as much context as possible. Providing examples and suggestions and copying information into the prompt or file was said to be crucial for receiving the most relevant and accurate suggestions from the GAI. I14 elaborated on a systematic approach to how to query: *"1. Provide context, 2. Ask clearly what you want and give detailed information about what you expect from the model's response, 3. Specify how you want it, 4. Never accept the first answer, it is an iterative process."* Another developer reflected on this systematic approach. Instead of asking for one solution to a specific problem, he asked for multiple solutions and strategies to solve the problem, allowing him to choose the best path forward.

4.6. Team dynamics

The reliance on GAI was said not to replace the human interactions within the team. I6 had a preference for human insights over AI for troubleshooting, emphasizing trust in colleagues. The informant stated, *"I prefer asking colleagues about issues I encounter. I trust them and don't need to fact-check the information they provide"*. This preference aligned with the observed patterns of peer-to-peer communication. Though the communication often related to personal topics, it revealed a tendency among team members to pose open questions to the group, inviting widespread engagement and leading to knowledge sharing. Such interactions frequently disrupted individual concentration in the team and mainly dealt with domain-specific issues. Supporting this observation, I4 stated, *"it's often easier to ask a colleague than to type into ChatGPT"*. Though there are exceptions for when working remotely, as I1 states *"If I'm at home, I might as well start by asking ChatGPT, since it takes longer to reach out to people"*.

However, some developers stated that they found it easy to use GAI as they did not have to interrupt anyone. Constantly bothering team members could be more difficult, whereas ChatGPT did not have a workflow to interrupt. Six interviewees mentioned that they considered team members' availability and time constraints when considering communicating with colleagues. I6 explained: *"I find it easy to over-rely on AI since it's always available. Constantly bothering my team members can be annoying, whereas ChatGPT doesn't have a workflow I interrupt."*

We observed that developers would primarily use Copilot or ChatGPT for *code* related issues. Most small issues could be solved by Copilot, as it had a general understanding of the context. *"While in the editor, if you are stuck on a problem, Copilot can often help you in the right direction"* (I14). During an observation, a developer asked Copilot *"what is the default value of a bool in GO?"*, which then returned an answer.

Another example, is that a developer was observed having issues with handling safe use of input fields in frontend. The developer first consulted with ChatGPT about HTML injections and other relevant themes. The developer got a lot of information and knowledge about the theme. After that, the developer asked a colleague about *domain* specific information and if they had some functions or components within the domain that he could use that was safe. In our case, the term *"domain"* refers to the knowledge, tools, practices, and components that are relevant and available within the organization. It includes predefined functions, components, design practices, APIs, and other resources that are developed to the organization's specific needs.

GAI tools often fall short on tasks requiring specific domain knowledge, leading developers to seek human expertise. For instance, when dealing with domain-specific issues, five interviewees mentioned that they prefer to consult a colleague first due to the lack of specialized information provided by GAI. One developer (I3) stated, *"For domain-related matters, I consult colleagues first as GAI lacks our specific knowledge. But for coding, I often use it."*

5. Discussion

The integration of GAI tools like Github Copilot has significantly altered the traditional developer workflow. Developers now spend less time typing code and more time reviewing AI-generated code suggestions. This shift has streamlined routine tasks, allowing developers to focus on higher-value activities. However, the

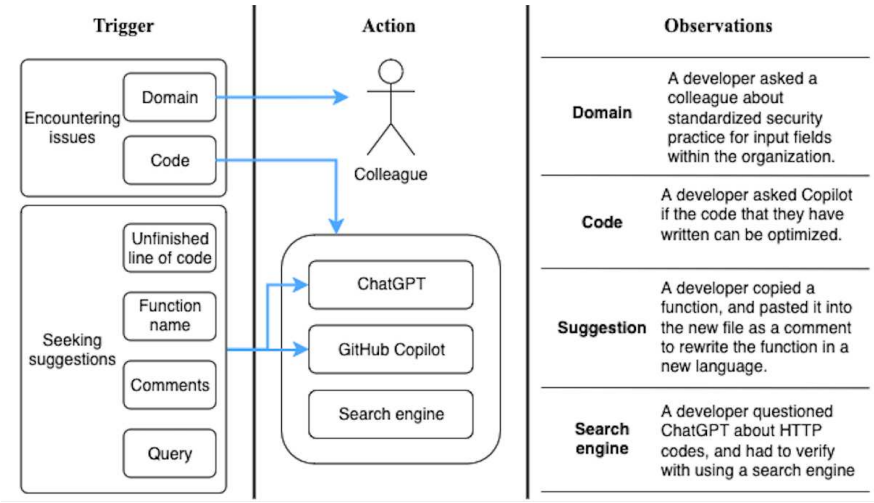

Figure 4. Development workflow with GAI tools

effectiveness of this new workflow hinges on the quality of AI suggestions, which often varies a lot. Further, when facing domain-specific questions or challenges, developers often turn to their colleagues for help. GAI tools lack the specialized knowledge and context needed to address issues unique to the organization's specific practices, tools, or codebases. For example, developers might consult colleagues about custom needs, or internal APIs that are not well-documented. The development workflow with GAI tools is shown in Figure 4.

Figure 4 shows that developers consult GAI for assistance with general coding questions, diminishing the frequency of peer-to-peer interactions. They would typically use AI as a first line of questioning for purely technical challenges. However, when faced with domain-specific questions, developers prioritize human expertise, addressing complexities that require a specified understanding of the context and domain.

We will now discuss the case in light of our research question: *How has the use of generative AI tools influenced a developer's workday?* To answer this question, we rely on the conceptual framework for good workdays described by Meyer et al. (2019), who identified factors impacting developers' assessment of a good workday.

Learning New Things. GAI tools have become a pivotal resource for developers seeking to learn new skills and knowledge quickly. By providing instant code suggestions and solutions, tools like ChatGPT have reduced the reliance on traditional search engines and forums. This convenience has made learning more efficient, as developers can obtain relevant information without extensive searches. They also noted that getting unstuck faster allowed them to tackle more challenging problems and learn more as a result.

Making Progress. The rapid retrieval of information and code suggestions allowed the developers to maintain momentum and make continuous progress. We found that experienced developers used GAI more efficiently compared to novice developers, which was also similar to the findings of Dakhel et al. (2023). Novice developers tended to spend more time reading and evaluating coding suggestions.

Getting progress with using Copilot and ChatGPT relies heavily on the users' ability to provide clear and accurate prompts (Imai, 2022; Yetistiren et al., 2022). Our findings show that the developers followed an iterative process to enhance each suggestion by adding more context and information. They started by providing minimal information to get an answer, and then they iteratively improved the prompt with more information, to make it clear and accurate. This negated the limitations for novice developers, who lacked the necessary skills to effectively utilize these tools. They risked integrating buggy or suboptimal code into their solutions, which could have affected the overall quality of their product.

For all groups of employees, GAI was found to help with progress. When developers make progress and spend their time efficiently, with few technical issues, the workday is perceived as good (Meyer et al., 2019). Despite this, the potential for incorrect suggestions means that developers must remain vigilant, balancing speed with accuracy. GAI use does not necessarily improve code quality Jaworski, Piotrkowski (2023).

Ability to Work Focused. GAI tools have a dual impact on developers' ability to maintain focus. On one hand, the tools streamline repetitive tasks, reducing

cognitive load and allowing developers to concentrate on more complex problems. On the other hand, frequent and sometimes irrelevant suggestions from tools like Copilot can be distracting, disrupting the coding flow. In pair programming sessions, the presence of a human partner often leads developers to prioritize human interaction over AI interaction, further highlighting the need for a balanced approach to using these tools.

Stress. The incorporation of GAI tools has the potential to reduce stress among developers by mitigating the mental burden associated with constant decision-making. Tools like Copilot provided a sense of direction and reduced uncertainty when coding, which gave the developers more confidence in their decisions. However, the need to verify and correct AI-generated suggestions can introduce a different kind of stress, particularly when dealing with inaccurate outputs. Overall, while GAI tools can enhance mental comfort, their reliability remains a critical factor in their impact on stress levels.

Productivity. GAI tools have generally been perceived as productivity enhancers within the software development process. By automating routine coding tasks and providing quick access to information, tools like ChatGPT and Copilot enable developers to work more efficiently. Peng et al. (2023) found that Copilot has a statistically and practically significant impact on productivity. The group that had access to GitHub Copilot was able to complete the task 55.8% faster than the control group. Such results were one reason for why CoPilot became such a hype as described in this study. However, developers in our study reported not gaining this productivity. When asking developers about the use of GAI tools, they claimed that it increased productivity by saving developers an average of 23 minutes per day.

The increased productivity is particularly evident in tasks that involve code autocompletion and syntax assistance. However, the productivity gains are tempered by the need to validate AI-generated suggestions and the occasional disruption caused by irrelevant inputs. Thus, while GAI tools hold promise for boosting productivity, their full potential is realized when combined with developer oversight and expertise.

A good workday. Implementing GAI tools into the development process had a significant effect on reducing mundane and repetitive tasks. Developers reported that these tools automated routine coding sequences. This shift allowed them to increase the duration of problem-solving of complex and rewarding tasks. Similarly, Ulfsnes et al. (2024) revealed that less time spent on manual, repetitive tasks led to increased productivity, motivation, and enjoyment. The factors affecting a good work day for developers include learning new things, making progress, and having the ability to work focused (Meyer et al., 2019). Our results suggest that implementing GAI tools supports a good workday.

6. Conclusion

The integration of generative AI (GAI) tools like Github Copilot has substantially transformed the traditional developer workflow, providing both significant benefits and challenges. These tools have streamlined routine tasks, allowing developers to focus on higher-value activities, thereby enhancing productivity and reducing cognitive load. However, the quality of AI-generated suggestions remains a critical factor, as inaccuracies necessitate careful validation and correction by developers.

While GAI tools are efficient for routine coding tasks, developers often prefer consulting colleagues for domain-specific issues and complex problem-solving, where human expertise and contextual understanding are indispensable. This balance between AI assistance and human collaboration is essential for maintaining workflow efficiency and accuracy.

Moreover, GAI tools have demonstrated potential in reducing stress by offering direction and reducing uncertainty in coding tasks. However, the necessity to verify AI outputs can introduce a different kind of stress, especially when suggestions are incorrect. Overall, GAI tools contribute to a more productive and satisfying workday by automating mundane tasks and facilitating continuous learning and progress. Nonetheless, their full potential is realized when combined with developer oversight and expertise, ensuring that the tools complement rather than disrupt the development process.

Future research should look at the long-term impacts of GAI integration on software development workflows. It should examine how sustained use of these tools influences developers' skills, particularly in terms of problem-solving and critical-thinking abilities. Additionally, there is a need to explore the potential shifts in team dynamics and collaboration, as well as the broader implications for organizational practices and software quality. Furthermore, investigating the ethical implications and establishing best practices for the responsible use of GAI in software development will be crucial for ensuring that these tools contribute positively to the industry.

Acknowledgments

We extend our sincere gratitude to Sparebank1 Utvikling, the survey and interview participants, and those who generously allowed us to observe their work. The Research Council of Norway supported this research through the 10xTeams project (grant 309344) and the TransformIT project (grant 321477).

References

Astromskis, S., Bavota, G., Janes, A., Russo, B., & Di Penta, M. (2017). Patterns of developers behaviour: A 1000-hour industrial study. *Journal of Systems and Software*, *132*, 85–97.

Berntzen, M., Stray, V., Moe, N. B., & Hoda, R. (2023). Responding to change over time: A longitudinal case study on changes in coordination mechanisms in large-scale agile. *Empirical Software Engineering*, *28*(5), 114. https://doi.org/10.1007/s10664-023-10349-0

Brynjolfsson, E., Li, D., & Raymond, L. R. (2023, April). *Generative ai at work* (Working Paper No. 31161). National Bureau of Economic Research. https://doi.org/10.3386/w31161

Cabrero-Daniel, B., Herda, T., Pichler, V., & Eder, M. (2024). Exploring human-ai collaboration in agile: Customised llm meeting assistants. *International Conference on Agile Software Development*, 163–178.

Chattopadhyay, S., Zimmermann, T., & Ford, D. (2021). Reel life vs. real life: How software developers share their daily life through vlogs. *Proceedings of the 29th ACM joint meeting on European SEconference and symposium on the foundations of SE*, 404–415.

Dakhel, A. M., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., & Jiang, Z. M. J. (2023). Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software*, *203*, 111734.

Dohmke, T. (2023, March). GitHub Copilot X: The AI-powered developer experience. Retrieved September 27, 2023, from https://github.blog/2023-03-22-github-copilot-x-the-ai-powered-developer-experience/

Gonçalves, M. K., de Souza, C. R., & Gonzalez, V. M. (2011). Collaboration, information seeking and communication: An observational study of software developers' work practices. *J. Univers. Comput. Sci.*, *17*(14), 1913–1930.

Gustavsson, T., Berntzen, M., & Stray, V. (2022). Changes to Team Autonomy in Large-Scale Software Development: A Multiple Case Study of Scaled Agile Framework (SAFe) Implementations. *International Journal of Information Systems and Project Management*, *10*(1), 29–46.

Imai, S. (2022). Is github copilot a substitute for human pair-programming? an empirical study. *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 319–321.

Jaworski, M., & Piotrkowski, D. (2023). Study of software developers' experience using the github copilot tool in the software development process.

Lazar Jonathan, H. H., Feng Jinjuan Heidi. (2017). *Research methods in human-computer interaction* (2nd ed.). Morgan Kaufmann.

Meyer, A. N., Barr, E. T., Bird, C., & Zimmermann, T. (2019). Today was a good day: The daily life of software developers. *IEEE Transactions on Software Engineering*, *47*(5), 863–880.

Miller, C., Rodeghero, P., Storey, M.-A., Ford, D., & Zimmermann, T. (2021). "how was your weekend?" software development teams working from home during covid-19. *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 624–636.

Nguyen-Duc, A., Cabrero-Daniel, B., Przybylek, A., Arora, C., Khanna, D., Herda, T., Rafiq, U., Melegati, J., Guerra, E., Kemell, K.-K., Saari, M., Zhang, Z., Le, H., Quan, T., & Abrahamsson, P. (2023). Generative Artificial Intelligence for Software Engineering – A Research Agenda.

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of ai on developer productivity: Evidence from github Copilot. *arXiv preprint arXiv:2302.06590*.

Russo, D., Baltes, S., van Berkel, N., Avgeriou, P., Calefato, F., Cabrero-Daniel, B., Catolino, G., Cito, J., Ernst, N., Fritz, T., et al. (2024). Generative ai in software engineering must be human-centered: The copenhagen manifesto. *Journal of Systems and Software*, 112115.

Sharp, H., Rogers, Y., & Preece, J. (2019). *Interaction design: Beyond human-computer interaction, fifth edition* (5th ed.). Wiley; Sons Canada, Limited, John.

Sofaer, S. (1999). Qualitative methods: What are they and why use them? *Health services research*, *34*(5 Pt 2), 1101.

Sridhara, G., G., R. H., & Mazumdar, S. (2023). Chatgpt: A Study on its Utility for Ubiquitous Software Engineering Tasks. *arXiv preprint arXiv:2305.16837*.

Stray, V., Moe, N. B., Bergersen, G., & Kirkerud, J. (2024). Behavioral aspects of agile software development: A case study on meeting practices in a fintech organization. *Proceeding of the 57th Hawaii International Conference on System Sciences (HICSS 2024)*.

Surameery, N. M. S., & Shakor, M. Y. (2023). Use chat gpt to solve programming bugs. *International Journal of Information Technology & Computer Engineering (IJITC) ISSN: 2455-5290*, *3*(01), 17–22.

Ulfsnes, R., Moe, N. B., Stray, V., & Skarpen, M. (2024). Transforming software development with generative ai: Empirical insights on collaboration and workflow. In A. Nguyen-Duc, P. Abrahamsson, & F. Khomh (Eds.), *Generative ai for effective software development* (pp. 219–234). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-55642-5_10

Wivestad, V., Barbala, A., & Stray, V. (2024). Copilot's island of joy: Balancing individual satisfaction with team interaction in agile development. *Agile Processes in Software Engineering and Extreme Programming - Workshops. 25th International Conference on Agile Software Development, XP 2024, Bolzano, Italy, June 4-7, 2024, Proceedings*.

Yetistiren, B., Ozsoy, I., & Tuzun, E. (2022). Assessing the quality of github copilot's code generation. *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering*, 62–71.

Yin, R. K. (2003). *Case study research and applications : Design and methods* (3rd. edition.). SAGE.

Zhang, B., Liang, P., Zhou, X., Ahmad, A., & Waseem, M. (2023). Practices and challenges of using github copilot: An empirical study. *Proceedings of the 35th International Conference on Software Engineering and Knowledge Engineering*.